

# Universidad Nacional Autónoma de México

## FACULTAD DE CIENCIAS, 2027-I

### COMPILADORES



---

## PRÁCTICA 01

# INFRAESTRUCTURA BÁSICA DEL ANALIZADOR LÉXICO DE MINIC

---

EQUIPO:

Delgado Díaz Hermes Alberto - 319258613

Medina Peralta Joaquín - 320202513

5 de septiembre de 2026

# 1. Objetivo

El objetivo de la práctica fue construir la infraestructura inicial de analizador léxico de MiniC.

La implementación debía conservar la posición de cada lexema, ignorar espacios en blanco, reconocer enteros y símbolos simples, reportar caracteres no reconocidos y producir `TOKEN_EOF` al finalizar el archivo.

También se buscó dejar una base modular para extender el lexer en la Práctica 2 sin concretar toda la lógica en `main.c`. Para ello se separaron las responsabilidades de lectura e integración, análisis léxico y representación de tokens.

# 2. Descripción de la solución

## 2.1. Organización modular

El proyecto sigue la estructura modular solicitada:

| Módulo                             | Responsabilidad                                                                                                              |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>src/main.c</code>            | Valida argumentos, abre el archivo, configura la salida en Windows, invoca <code>lexer_scan</code> y libera/cierra recursos. |
| <code>include/lexer/token.h</code> | Define <code>TokenType</code> , <code>Token</code> y la interfaz para administrar tokens.                                    |
| <code>src/lexer/token.c</code>     | Crea, imprime y destruye tokens; traduce cada <code>TokenType</code> a su nombre textual                                     |
| <code>include/lexer/lexer.h</code> | Declara la interfaz pública del analizador léxico.                                                                           |
| <code>src/lexer/lexer.c</code>     | Recorre el archivo, actualiza línea/columna y reconoce los elementos léxicos de esta práctica.                               |
| <code>tests/run_tests.sh</code>    | Ejecuta casos públicos/proprios y compara exactamente la salida con los resultados esperados.                                |

`main.c` se mantiene como punto de integración y no contiene las reglas de reconocimiento. La lógica léxica reside en `lexer.c`, mientras que la representación y administración de memoria de cada token reside en `token.c`.

## 2.2. Representación de tokens

Se utilizó la estructura sugerida por el enunciado:

```
typedef struct {
    TokenType type;
    char *lexeme;
    size_t line;
    size_t column;
}
```

```
} Token;
```

El lexema se almacena como una cadena reservada dinámicamente (malloc dentro de `token_init`, a partir de una copia del texto reconocido) y se libera explícitamente con `token_destroy` inmediatamente después de imprimir cada token. Con esto se evita mantener en memoria más de un lexema a la vez y se evitan fugas de memoria.

### 2.3. Recorrido y reconocimiento

El analizador utiliza un único bucle que lee un carácter a la vez con `fgetc` y decide, en cada iteración, a qué categoría pertenece:

- si es un salto de línea (`\n`, `\r` o la secuencia `\r\n`) o un espacio/tabulador, únicamente actualiza la posición y continúa sin generar token
- si es un dígito, delega en una función auxiliar (`scan_integer_lexeme`) que consume todos los dígitos consecutivos siguientes usando `fgetc/ungetc`, de modo que el primer carácter que no es dígito queda disponible para la siguiente iteración del bucle principal
- si es alguno de los símbolos simples reconocidos (`simple_token_type`), genera el token de un carácter correspondiente
- en cualquier otro caso, genera un token `ERROR` con el carácter, línea y columna actuales, y continúa con el siguiente carácter.

Al llegar a EOF se genera el token `TOKEN_EOF` con la posición final acumulada (después de haber procesado, y por tanto ya haber avanzado la posición sobre, cualquier espacio en blanco final).

### 2.4. Seguimiento de línea y columna

Las líneas inician en 1 y las columnas en 0, como exige la especificación. La función `advance_position` centraliza la actualización de la posición: un carácter ordinario (incluyendo espacio y tabulador) incrementa la columna en uno; `\n` incrementa la línea y reinicia la columna a cero; `\r` hace lo mismo, y además consulta el siguiente carácter del archivo (`fgetc`) para, si es `\n`, consumirlo como parte del mismo salto de línea (tratando `\r\n` como un único salto); si no lo es, lo regresa al flujo con `ungetc` para que sea procesado normalmente en la siguiente iteración.

### 2.5. Formato de salida y errores operativos

Los tokens se imprimen en `stdout` en el orden de aparición. Los errores de uso, apertura, lectura, cierre o reserva de memoria se imprimen en `stderr`. Esta separación evita contaminar la secuencia de tokens utilizada por las pruebas automáticas.

```

1:0 INTEGER 10
1:3 PLUS +
1:5 INTEGER 20
1:7 SEMICOLON ;
1:8 TOKEN_EOF

```

### 3. Resultados y pruebas

La implementación se verificó contra las 10 pruebas públicas proporcionadas y contra 4 pruebas adicionales diseñadas por el equipo, comparando la salida estándar exacta contra el archivo `.out` esperado mediante el script `tests/run_tests.sh` (ejecutable también como `make test`).

| Suite              | Casos | Resultado      |
|--------------------|-------|----------------|
| Pruebas públicas   | 10    | 10/10 exitosas |
| Pruebas del equipo | 4     | 4/4 exitosas   |

### 4. Problemas encontrados

El principal problema de portabilidad fue que los finales de línea no se comportan igual en todos los sistemas. La práctica exige distinguir correctamente los bytes de entrada y, al mismo tiempo, producir una salida exacta para poder compararla contra archivos `.out`.

#### 4.1. Traducción de la entrada en modo texto

En Windows, abrir un archivo en modo texto puede provocar traducciones automáticas de finales de línea. Si el runtime transforma CRLF antes de que el lexer lo observe, el programa deja de analizar los bytes originales. Para evitar esa dependencia del sistema operativo, el archivo fuente se abre con modo binario:

```
file = fopen(argv[1], "rb");
```

En Linux el modo binario no cambia el comportamiento respecto al modo texto, por lo que esta decisión mantiene una entrada consistente en ambos entornos.

#### 4.2. Traducción de stdout en Windows

Un segundo problema aparece al imprimir. En Windows, stdout en modo texto puede transformar cada LF escrito por printf en CRLF. A simple vista el resultado parece correcto, pero diff puede detectar un CR adicional frente a los `.out` almacenados con LF. La solución incluida se activa solo bajo `_WIN32`:

```
#ifdef _WIN32
_setmode( _fileno( stdout ), _O_BINARY );
#endif
```

Así la salida conserva exactamente los bytes LF generados por `token_print`. En Linux este bloque ni siquiera se compila.

### 4.3. Git y `core.autocrlf`

Git puede convertir CRLF y LF durante checkout/commit según la configuración `core.autocrlf`. Esto es especialmente peligroso aquí porque algunas entradas `.mc` contienen CRLF de manera intencional para probar el lexer. Una normalización automática podría cambiar la prueba sin que el código haya cambiado.

Se agregó `.gitattributes` para mantener código, scripts y archivos esperados con LF, pero marcar los `fixtures tests/*/inputs/*.mc` como no textuales, preservando sus bytes. También se fuerza LF para `run_tests.sh`, evitando errores típicos en Linux cuando un script termina en CRLF (por ejemplo, un intérprete leído como `bash\r`).

### 4.4. Herramientas Unix en Windows

El Makefile y el runner utilizan bash, diff, rm y mkdir. Por ello, la automatización se ejecuta directamente en Linux/WSL y en entornos Windows que proporcionan herramientas POSIX, como MSYS2/MinGW64.

## 5. Conclusiones

La práctica permitió construir una base funcional y modular para el analizador léxico de MiniC. El programa ya transforma un archivo fuente en tokens básicos, conserva la ubicación exacta de cada lexema, maneja errores sin detener el recorrido y representa correctamente el final del archivo.

Uno de los aprendizajes más relevantes fue que el análisis léxico no depende únicamente de reconocer caracteres: también exige controlar cuidadosamente el estado del flujo, la memoria y detalles de portabilidad como CR, LF y CRLF. Estos detalles son importantes porque el formato de salida se evalúa de manera exacta.

## 6. Uso de herramientas de Inteligencia Artificial

Herramienta: Claude.

Propósito: Apoyo para resolver dudas de compatibilidad entre Windows y Linux, principalmente sobre finales de línea LF (`\n`), CRLF (`\r\n`) y CR (`\r`), además del manejo de espacios en blanco y tabulaciones.

Prompt representativo: "Ayúdame a revisar los problemas de compatibilidad entre Windows y Linux, especialmente el manejo de `\r\n`, `\n`, `\r`, los espacios en blanco y los tabuladores."

Tipo de apoyo: Explicación técnica sobre diferencias de finales de línea entre sistemas operativos, normalización de archivos y conservación de espacios/tabulaciones en las pruebas

Revisión/adaptación: La información proporcionada se revisó y se adaptó a los requisitos de la práctica. Las decisiones finales se contrastaron con el comportamiento del código y con los casos de prueba utilizados en la entrega.

## 7. Referencias

- <https://www.webtutoriales.com/articulos/2025/05/08/lf-crlf/>
- <https://www.msys2.org/docs/environments/>
- <https://devdocs.io/c/>
- Pablo Martínez, Y. J.; Salazar González, P. Y. (2026). Compiladores - Práctica 1, semestre 2027-1. Facultad de Ciencias, UNAM.